

Railroad Pier-Cap Strut-and-Tie Design

This production-oriented notebook generates and checks a longitudinal two-dimensional STM for a pile-supported concrete railroad bridge cap. It uses one common physical candidate structure for all supplied combinations and automates conservative strut, tie, nodal-zone, anchorage, and section-level reinforcement checks.

Important status

- The included load factors are demonstration values, **not AREMA design combinations**.
- Enter project-specific combinations from the governing AREMA edition.
- Positive member force denotes tension; negative force denotes compression.
- AREMA 2025 Article 2.30.2b resistance factors, Article 2.42 nodal factors, and the supplied reinforcement provisions are implemented.
- Final three-dimensional bar positioning, drawings, foundation design, and project load generation are outside this two-dimensional analysis.
- Arbitrary polygonal cross-section and local ledge analysis are outside this notebook's scope.

```
In [1]: from pathlib import Path
import sys

import matplotlib
matplotlib.use('module://matplotlib_inline.backend_inline')
import matplotlib.pyplot as plt
import numpy as np
from IPython.display import display

repo_root = next(
    path for path in (Path.cwd(), *Path.cwd().parents)
    if (path / 'stm_solver').exists() and (path / 'notebooks' / 'pier_cap_design.ip
)
if str(repo_root) not in sys.path:
    sys.path.insert(0, str(repo_root))

from stm_solver.design_checks import (
    apply_conservative_strut_policy, arema_2025_nodal_stress_factors, arema_2025_st
    REBAR_TABLE, calculate_standard_hook_development,
    calculate_tension_development_length,
    calculate_tie_end_anchorage_geometry, check_reinforcement_cage_congestion,
    check_strut, check_interior_nodal_zone_faces, classify_strut_shapes, design_ort
    check_provided_crack_control_grid, check_provided_tie_bar_layout,
    design_tie_bar_layout, extended_strut_tie_critical_section,
    check_subdivided_nodal_zone_faces, check_tie,
    construct_continuous_tie_nodal_zone,
    construct_single_strut_external_nodal_zone,
)
```

```

from stm_solver.ground_structure import (
    find_member_crossings, optimize_frame_boundary_load_cases,
    optimize_pier_cap_load_cases,
)
from stm_solver.pier_cap import (
    FoundationSupport, LoadCombination, LoadComponent, LoadPointForce,
    PierCapGeometry, PierCapLoadSet, PierCapModel,
)
from stm_solver.tables import (
    conservative_strut_design_records, crack_control_records,
    design_check_records, export_csv_tables, export_excel_tables, export_json_table,
    strut_shape_records, strut_width_records, tie_anchorage_records,
    reinforcement_cage_review_records,
    standard_hook_records, tension_development_records,
    tie_bar_layout_records,
    records_to_dataframes,
)
from stm_solver.reporting import (
    build_pier_cap_calculation_report, build_pier_cap_html_report,
    write_calculation_report, write_html_report,
)
from stm_solver.versioning import build_calculation_metadata

```

1. User inputs

Edit only this section for a new demonstration model. Keep all coordinates in inches and forces in kips. Load-point IDs must retain the same coordinates in every component in which they appear.

```

In [2]: project = {
    'name': 'FHWA NHI-13-0126 Example 3 - Production Notebook Validation',
    'designer': 'Validation benchmark', 'date': '09/23/2026', 'revision': '1',
    'governing_standard': 'FHWA-NHI-13-012, Design Example 3',
    'units': {'length': 'ft', 'force': 'kip', 'stress': 'ksi'},
}
cap = {'length': 47.5, 'depth': 5.12, 'width': 4.0, 'top_tie_y': 5.62, 'bottom_tie_y': 0.0}
materials = {'fc_ksi': 4.0, 'fy_ksi': 60.0}
detailing = {'clear_cover': 2.0, 'maximum_aggregate_size': 0.75, 'lightweight_concrete': False}
provided_reinforcement = None
arema_phi = arema_2025_stm_resistance_factors()
code_profile = {'name': 'FHWA Example 3 force-flow validation only', 'phi_tie': arema_phi}
output_options = {'write_files': False, 'directory': repo_root / 'validation' / 'files'}
files = []
load_components = {}
load_face_widths = {}
load_combinations = {}
generation = {'min_angle': 25.0, 'max_angle': 90.0, 'force_tolerance': 0.001}
analysis_mode = 'frame_boundary'
frame_model = {
    'top_nodes': [
        {'id': 'A', 'x': 0.57, 'y': 5.62}, {'id': 'B', 'x': 12.83, 'y': 5.62}, {'id': 'C', 'x': 25.66, 'y': 5.62},
        {'id': 'D', 'x': 29.87, 'y': 5.62}, {'id': 'E', 'x': 38.29, 'y': 5.62}, {'id': 'F', 'x': 47.5, 'y': 5.62}
    ],
    'bottom_nodes': [
        {'id': 'A', 'x': 0.57, 'y': 0.0}, {'id': 'B', 'x': 12.83, 'y': 0.0}, {'id': 'C', 'x': 25.66, 'y': 0.0},
        {'id': 'D', 'x': 29.87, 'y': 0.0}, {'id': 'E', 'x': 38.29, 'y': 0.0}, {'id': 'F', 'x': 47.5, 'y': 0.0}
    ],
    'members': [
        {'id': 'AB', 'x1': 0.57, 'y1': 5.62, 'x2': 12.83, 'y2': 5.62}, {'id': 'BC', 'x1': 12.83, 'y1': 5.62, 'x2': 25.66, 'y2': 5.62},
        {'id': 'CD', 'x1': 25.66, 'y1': 5.62, 'x2': 29.87, 'y2': 5.62}, {'id': 'DE', 'x1': 29.87, 'y1': 5.62, 'x2': 38.29, 'y2': 5.62},
        {'id': 'EF', 'x1': 38.29, 'y1': 5.62, 'x2': 47.5, 'y2': 5.62},
        {'id': 'AD', 'x1': 0.57, 'y1': 5.62, 'x2': 29.87, 'y2': 0.0}, {'id': 'BE', 'x1': 12.83, 'y1': 5.62, 'x2': 38.29, 'y2': 0.0},
        {'id': 'CF', 'x1': 25.66, 'y1': 5.62, 'x2': 47.5, 'y2': 0.0}
    ],
    'load_combos': [
        {'id': '1', 'x': 0.57, 'y': 5.62, 'force': 100, 'angle': 0},
        {'id': '2', 'x': 12.83, 'y': 5.62, 'force': 100, 'angle': 0},
        {'id': '3', 'x': 25.66, 'y': 5.62, 'force': 100, 'angle': 0},
        {'id': '4', 'x': 29.87, 'y': 5.62, 'force': 100, 'angle': 0},
        {'id': '5', 'x': 38.29, 'y': 5.62, 'force': 100, 'angle': 0},
        {'id': '6', 'x': 47.5, 'y': 5.62, 'force': 100, 'angle': 0}
    ],
    'load_faces': [
        {'id': '1', 'x1': 0.57, 'y1': 5.62, 'x2': 12.83, 'y2': 5.62, 'width': 4.0},
        {'id': '2', 'x1': 12.83, 'y1': 5.62, 'x2': 25.66, 'y2': 5.62, 'width': 4.0},
        {'id': '3', 'x1': 25.66, 'y1': 5.62, 'x2': 29.87, 'y2': 5.62, 'width': 4.0},
        {'id': '4', 'x1': 29.87, 'y1': 5.62, 'x2': 38.29, 'y2': 5.62, 'width': 4.0},
        {'id': '5', 'x1': 38.29, 'y1': 5.62, 'x2': 47.5, 'y2': 5.62, 'width': 4.0}
    ],
    'load_combos': [
        {'id': '1', 'x': 0.57, 'y': 5.62, 'force': 100, 'angle': 0},
        {'id': '2', 'x': 12.83, 'y': 5.62, 'force': 100, 'angle': 0},
        {'id': '3', 'x': 25.66, 'y': 5.62, 'force': 100, 'angle': 0},
        {'id': '4', 'x': 29.87, 'y': 5.62, 'force': 100, 'angle': 0},
        {'id': '5', 'x': 38.29, 'y': 5.62, 'force': 100, 'angle': 0},
        {'id': '6', 'x': 47.5, 'y': 5.62, 'force': 100, 'angle': 0}
    ],
    'load_faces': [
        {'id': '1', 'x1': 0.57, 'y1': 5.62, 'x2': 12.83, 'y2': 5.62, 'width': 4.0},
        {'id': '2', 'x1': 12.83, 'y1': 5.62, 'x2': 25.66, 'y2': 5.62, 'width': 4.0},
        {'id': '3', 'x1': 25.66, 'y1': 5.62, 'x2': 29.87, 'y2': 5.62, 'width': 4.0},
        {'id': '4', 'x1': 29.87, 'y1': 5.62, 'x2': 38.29, 'y2': 5.62, 'width': 4.0},
        {'id': '5', 'x1': 38.29, 'y1': 5.62, 'x2': 47.5, 'y2': 5.62, 'width': 4.0}
    ]
}

```



```

load_set.validate_envelope(geometry)
assembled_loads = load_set.assemble_all()
models = {name: PierCapModel(geometry, loads, supports, metadata=project) for name,
combination_loads = {name: [load.to_solver_dict() for load in loads] for name,
piles = [support.to_solver_dict() for support in supports]
print(f"Validated {len(supports)} supports, {len(components)} load components,
else:
    if frame_model is None:
        raise ValueError("frame_model is required when analysis_mode='frame_boundar
frame_nodes = frame_model['top_nodes'] + frame_model['lower_nodes'] + frame_mod
frame_node_lookup = {node['id']: node for node in frame_nodes}
combination_loads = {
    name: [dict(load, id=load['node'], x=frame_node_lookup[load['node']]['x'],
    for name, loads in frame_model['load_cases'].items()
}
piles = [dict(node, type='fixed', face_width=0.0) for node in frame_model['boun
models = {}
print(f"Validated {len(frame_model['boundary_nodes'])} frame-boundary nodes and
for name, loads in combination_loads.items():
    resultant = np.sum([[load.get('Px', 0.0), load.get('Py', 0.0)] for load in load
print(f" {name:<30} Fx={resultant[0]:9.1f} kip Fy={resultant[1]:9.1f} kip")

```

Validated 4 frame-boundary nodes and 1 load case(s).

FHWA_EXAMPLE_3 Fx= 0.0 kip Fy= -1553.7 kip

3. Generate and solve one common STM ground structure

One candidate structure is generated from the common physical load/support locations.

Candidates unused by every combination are pruned, every case is re-solved for equilibrium, and force reversals are reported explicitly.

```

In [4]: if run_design_checks:
        common_analysis = optimize_pier_cap_load_cases(
            combination_loads, piles,
            top_tie_y=cap['top_tie_y'], bottom_tie_y=cap['bottom_tie_y'],
            min_angle=generation['min_angle'], max_angle=generation['max_angle'],
        )
    else:
        common_analysis = optimize_frame_boundary_load_cases(
            frame_model['top_nodes'], frame_model['lower_nodes'], frame_model['boundary
            frame_model['load_cases'],
            boundary_member_targets=frame_model.get('boundary_member_targets'),
            target_relative_tolerance=frame_model.get('target_relative_tolerance', 0.0)
            min_angle=generation['min_angle'], max_angle=generation['max_angle'],
            force_tolerance=generation.get('force_tolerance', 1e-6),
        )
    results = common_analysis.case_results

    summary_records = []
    for name, result in results.items():
        summary_records.append({
            'combination': name, 'candidate_count': result.candidate_count,
            'selected_count': len(result.member_forces),

```

```

        'equilibrium_residual_kip': result.equilibrium_residual,
    })
    assert result.equilibrium_residual < 1e-7
    result_tables = {name: records_to_dataframes(ground_structure_records(result)) for
summary_table = records_to_dataframes({'summary': summary_records})['summary']}
    common_tables = records_to_dataframes(multi_load_ground_structure_records(common_an
display(summary_table)
    print(f'Common candidates: {common_analysis.initial_candidate_count} generated, {1e
if common_analysis.force_reversals:
        display(common_tables['force_reversals'])
    else:
        print('Force-reversal check: PASS - no common member changes between tension an

```

	combination	candidate_count	selected_count	equilibrium_residual_kip
0	FHWA_EXAMPLE_3	29	29	2.273737e-13

Common candidates: 29 generated, 29 retained after 0 pruning iteration(s).
Force-reversal check: PASS - no common member changes between tension and compression.

```

In [5]: crossing_failures = {}
        for name, result in results.items():
            crossings = find_member_crossings(
                result.truss_model.nodes, result.truss_model.members,
            )
            if crossings:
                crossing_failures[name] = crossings
        if crossing_failures:
            details = '; '.join(
                f'{name}: member-index pairs {pairs}'
                for name, pairs in crossing_failures.items()
            )
            raise RuntimeError(
                'Topology review failed: unintended non-nodal member crossings were found.'
                + details
            )
        print('Crossing check: PASS - no non-nodal selected-member intersections.')

```

Crossing check: PASS - no non-nodal selected-member intersections.

```

In [6]: if run_design_checks:
        print(f"{'combination':<30} {'support':<10} {'Rx (kip)':>12} {'Ry (kip)':>12}")
        print('-' * 68)
        inactive_supports = []
        for name, result in results.items():
            for pile_index, pile in enumerate(piles):
                node = result.truss_model.support_node_ids[pile_index]
                rx = result.reactions.get(2 * node, 0.0)
                ry = result.reactions.get(2 * node + 1, 0.0)
                print(f"{'name':<30} {'pile[id]':<10} {rx:12.1f} {ry:12.1f}")
                if abs(rx) < 1e-6 and abs(ry) < 1e-6:
                    inactive_supports.append((name, pile[id]))
        else:
            print(f"{'combination':<30} {'boundary':<10} {'Rx (kip)':>12} {'Ry (kip)':>12}")
            print('-' * 68)

```

```

offset = len(frame_model['top_nodes']) + len(frame_model['lower_nodes'])
for name, result in results.items():
    for boundary_index, boundary in enumerate(frame_model['boundary_nodes']):
        node = offset + boundary_index
        print(f"{name:<30} {boundary['id']:<10} {result.reactions.get(2*node, 0)}")

```

combination	boundary	Rx (kip)	Ry (kip)
FHWA_EXAMPLE_3	A_prime	-1.3	-255.8
FHWA_EXAMPLE_3	G_prime	260.8	838.8
FHWA_EXAMPLE_3	L_prime	-260.8	1177.4
FHWA_EXAMPLE_3	F_prime	1.3	-206.6

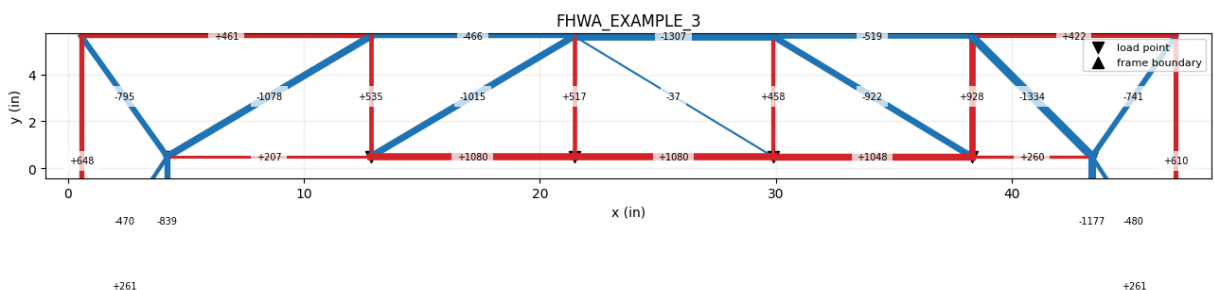
4. Topology and force review

Line width is proportional to force magnitude. Red members are ties; blue members are struts. This plot is a required engineering review gate—not merely presentation output.

```

In [7]: fig, axes = plt.subplots(len(results), 1, figsize=(13, 4.2 * len(results)), squeeze=False)
for ax, (name, result) in zip(axes[:, 0], results.items()):
    truss = result.truss_model
    scale = max(abs(result.member_forces))
    for (i, j), kind, force in zip(truss.members, truss.member_types, result.member_forces):
        color = '#d62728' if kind == 'tie' else '#1f77b4'
        width = 1.4 + 5.0 * abs(force) / scale
        ax.plot(truss.nodes[[i, j], 0], truss.nodes[[i, j], 1], color=color, lw=width)
        midpoint = (truss.nodes[i] + truss.nodes[j]) / 2
        ax.text(*midpoint, f'{force:+.0f}', fontsize=7, ha='center', va='center',
                bbox={'facecolor': 'white', 'alpha': 0.75, 'edgecolor': 'none'})
    loads = combination_loads[name]
    ax.scatter([load['x'] for load in loads], [load['y'] for load in loads],
               marker='v', s=75, color='black', label='load point')
    ax.scatter([pile['x'] for pile in piles], [pile['y'] for pile in piles],
               marker='^', s=75, color='black', label=('pile/support' if run_design else 'pile'))
    ax.set(title=name, xlabel='x (in)', ylabel='y (in)', aspect='equal',
           xlim=(-0.02 * cap['length'], 1.02 * cap['length']),
           ylim=(-0.08 * cap['depth'], 1.12 * cap['depth']))
    ax.grid(alpha=0.18)
    ax.legend(loc='upper right', fontsize=8)
fig.tight_layout()
display(fig)
plt.close(fig)

```



```

In [8]: def member_signature(result, member):
        i, j = member

```

```

points = [
    tuple(float(value) for value in np.round(result.truss_model.nodes[index], 6
        for index in (i, j)
    ]
    return tuple(sorted(points))

topologies = {
    name: {member_signature(result, member) for member in result.truss_model.member
        for name, result in results.items()}
}
common_members = set.intersection(*topologies.values())
union_members = set.union(*topologies.values())
print(f"Members common to every combination: {len(common_members)}")
print(f"Members appearing in any combination: {len(union_members)}")
if common_members != union_members:
    print('REVIEW REQUIRED: independently optimized combinations produced different
else:
    print('All demonstration combinations selected the same topology.')

```

Members common to every combination: 29

Members appearing in any combination: 29

All demonstration combinations selected the same topology.

5. Tie, strut, and nodal-zone checks

The calculations use the supplied AASHTO strut equation, AREMA 2025 Article 2.30.2b resistance factors, Article 2.42 face-specific nodal factors, and the supplied AREMA reinforcement development/detailing provisions. Consequential project defaults remain visible above.

```

In [9]: if run_design_checks:
        # Provisional design inputs - review and replace when AREMA criteria are supplied
        preliminary_design = {
            'phi_tie': code_profile['phi_tie'],
            'phi_concrete': code_profile['phi_concrete'],
            'strut_eps_1': 0.002,
            'strut_fcu_cap_factor': 0.85,
            'node_stress_factors': arema_2025_nodal_stress_factors(materials['fc_ksi']),
            'nodal_confinement_factor': 1.0,
            'load_nodal_back_depth': 2.0 * (cap['depth'] - cap['top_tie_y']),
            'support_nodal_back_depth': 2.0 * cap['bottom_tie_y'],
            'crack_control_bar_size': 5,
            'crack_control_legs': 2,
            'crack_control_ratio': 0.003,
            'crack_control_max_spacing': 12.0,
            'crack_control_spacing_increment': 0.5,
            'maximum_aggregate_size': detailing['maximum_aggregate_size'],
            'maximum_longitudinal_bar_size': detailing['maximum_longitudinal_bar_size'],
            'crack_control_effective_depth': max(
                cap['top_tie_y'], cap['depth'] - cap['bottom_tie_y'],
            ),
            'lightweight_concrete': detailing['lightweight_concrete'],
            'epoxy_coated_reinforcement': detailing['epoxy_coated_reinforcement'],
            'credit_excess_reinforcement_for_development': False,

```

```

}
load_point_ids = sorted({load['id'] for loads in load_components.values() for l
load_bearing_widths = dict(load_face_widths)
pile_bearing_widths = {pile['id']: pile['face_width'] for pile in piles}
print(code_profile['name'])
print(code_profile['resistance_factor_status'])
print(f"Detailing defaults: cover={detailing['clear_cover']} in, aggregate={det
else:
print('Frame-boundary mode: bearing, anchorage, and member-capacity detailing c

```

Frame-boundary mode: bearing, anchorage, and member-capacity detailing checks are outside this validation scope.

```

In [10]: if run_design_checks:
force_envelope = {}
for combination, result in results.items():
    for member, force in zip(result.truss_model.members, result.member_forces):
        signature = member_signature(result, member)
        record = force_envelope.setdefault(signature, {
            'max_tension': 0.0, 'max_compression': 0.0,
            'tension_combination': None, 'compression_combination': None,
        })
        if force > record['max_tension']:
            record['max_tension'] = float(force)
            record['tension_combination'] = combination
        if force < record['max_compression']:
            record['max_compression'] = float(force)
            record['compression_combination'] = combination
    print(f'Force envelope assembled for {len(force_envelope)} geometric members.')
else:
print('Frame-boundary mode: bearing, anchorage, and member-capacity detailing c

```

Frame-boundary mode: bearing, anchorage, and member-capacity detailing checks are outside this validation scope.

```

In [11]: if run_design_checks:
tie_checks = []
for member_id, (signature, record) in enumerate(sorted(force_envelope.items())):
    if record['max_tension'] <= 0.0:
        continue
    check = check_tie(
        member_id, record['max_tension'], materials['fy_ksi'],
        phi=preliminary_design['phi_tie'],
    )
    tie_checks.append((signature, record['tension_combination'], check))
print(f"{'member endpoints (in)':<38} {'combo':<26} {'Tu':>8} {'As req':>9} {'p
print('-' * 108)
for signature, combination, check in tie_checks:
    label = f'{signature[0]} to {signature[1]}'
    provided = f'{check.n_bars}-#{check.bar_size}'
    print(f"{label:<38} {combination:<26} {check.force:8.1f} {check.As_required
tie_bar_layouts = []
for signature, combination, check in tie_checks:
    layout = design_tie_bar_layout(
        check.As_required, cap['width'],
        tie_zone_depth=min(
            preliminary_design['load_nodal_back_depth'],

```

```

        preliminary_design['support_nodal_back_depth'],
    ),
    enclosure_bar_size=preliminary_design['crack_control_bar_size'],
    clear_cover=detailing['clear_cover'],
    maximum_aggregate_size=preliminary_design['maximum_aggregate_size'],
    allowed_bar_sizes=tuple(range(detailing['minimum_longitudinal_bar_size']
    )
    tie_bar_layouts.append((signature, combination, layout))
tie_layout_records = tie_bar_layout_records(
    [layout for _, _, layout in tie_bar_layouts]
)
for record, (signature, combination, _) in zip(tie_layout_records, tie_bar_layo
    record.update({'member_endpoints': signature, 'combination': combination})
tie_layout_table = records_to_dataframes({'tie_layouts': tie_layout_records})['
print(f"{'member endpoints (in)':<38} {'combo':<20} {'As req':>8} {'layout':>15}
print('-' * 105)
for signature, combination, layout in tie_bar_layouts:
    label = f'{signature[0]} to {signature[1]}'
    arrangement = f"{layout.bar_count}-#{layout.bar_size} / {layout.layers} lay
    print(f"{label:<38} {combination:<20} {layout.as_required:8.3f} {arrangemen
layer_demands = {}
for signature, combination, check in tie_checks:
    y0, y1 = signature[0][1], signature[1][1]
    if not np.isclose(y0, y1):
        continue
    layer = 'top longitudinal tie' if np.isclose(y0, cap['top_tie_y']) else 'bo
    governing = layer_demands.get(layer)
    if governing is None or check.As_required > governing['As_required']:
        layer_demands[layer] = {
            'As_required': check.As_required, 'combination': combination,
            'member_endpoints': signature,
        }
common_tie_layouts = {}
print('\nCommon longitudinal layer schedule:')
for layer, demand in layer_demands.items():
    short_name = 'top' if layer.startswith('top') else 'bottom'
    provided = (provided_reinforcement or {}).get(short_name)
    layout_options = {
        'tie_zone_depth': min(preliminary_design['load_nodal_back_depth'], prel
        'enclosure_bar_size': preliminary_design['crack_control_bar_size'],
        'clear_cover': detailing['clear_cover'],
        'maximum_aggregate_size': preliminary_design['maximum_aggregate_size'],
    }
    if provided is None:
        layout = design_tie_bar_layout(
            demand['As_required'], cap['width'], **layout_options,
            allowed_bar_sizes=tuple(range(detailing['minimum_longitudinal_bar_s
        )
    else:
        layout = check_provided_tie_bar_layout(
            demand['As_required'], cap['width'], **provided, **layout_options,
        )
    common_tie_layouts[layer] = layout
    print(f" {layer}: {layout.bar_count}-#{layout.bar_size} in {layout.layers}
tie_layer_positions = {layer: set() for layer in common_tie_layouts}
for signature, _, _ in tie_checks:

```

```

y0, y1 = signature[0][1], signature[1][1]
if not np.isclose(y0, y1):
    continue
layer = 'top longitudinal tie' if np.isclose(y0, cap['top_tie_y']) else 'bottom longitudinal tie'
tie_layer_positions[layer].update((signature[0][0], signature[1][0]))
provided_grid = (provided_reinforcement or {}).get('crack_control')
if provided_grid is None:
    crack_control_grid = design_orthogonal_crack_control_grid(
        cap['width'], preliminary_design['crack_control_effective_depth'],
        bar_size=preliminary_design['crack_control_bar_size'],
        legs=preliminary_design['crack_control_legs'],
        required_ratio=preliminary_design['crack_control_ratio'],
        absolute_spacing_limit=preliminary_design['crack_control_max_spacing'],
        spacing_increment=preliminary_design['crack_control_spacing_increment']
    )
else:
    crack_control_grid = check_provided_crack_control_grid(
        cap['width'], preliminary_design['crack_control_effective_depth'],
        provided_grid, required_ratio=preliminary_design['crack_control_ratio'],
        absolute_spacing_limit=preliminary_design['crack_control_max_spacing'],
    )
development_grid = crack_control_grid[0]
tie_development_calculations = {}
tie_hook_alternatives = {}
tie_anchorage_geometries = []
for layer, layout in common_tie_layouts.items():
    center_cover = layout.clear_cover + REBAR_TABLE[layout.enclosure_bar_size]
    development = calculate_tension_development_length(
        layout.bar_size, materials['fy_ksi'], materials['fc_ksi'],
        top_bar=layer.startswith('top'),
        lightweight=preliminary_design['lightweight_concrete'],
        epoxy_coated=preliminary_design['epoxy_coated_reinforcement'],
        cover=center_cover, clear_spacing=layout.transverse_clear_spacing,
        transverse_reinforcement_area=development_grid.steel_area_per_spacing,
        transverse_spacing=development_grid.selected_spacing,
        bars_in_splitting_plane=layout.bar_count,
        as_required=layout.as_required, as_provided=layout.as_provided,
        apply_excess_reinforcement_factor=preliminary_design['credit_excess_reinforcement']
    )
    tie_development_calculations[layer] = development
    hook = calculate_standard_hook_development(
        layout.bar_size, materials['fy_ksi'], materials['fc_ksi'],
        hook_angle=90,
        epoxy_coated=preliminary_design['epoxy_coated_reinforcement'],
        adequate_cover=(layout.clear_cover + REBAR_TABLE[layout.enclosure_bar_size]),
        lightweight=preliminary_design['lightweight_concrete'],
    )
    hook_fits_transversely = hook.transverse_projection <= cap['depth'] - 2.0 *
    tie_hook_alternatives[layer] = (hook, hook_fits_transversely)
    preliminary_geometry = calculate_tie_end_anchorage_geometry(
        layer, tie_layer_positions[layer], 0.0, cap['length'], layout.bar_size,
        enclosure_bar_size=preliminary_design['crack_control_bar_size'],
        clear_cover=detailing['clear_cover'],
        required_development_lengths={'left': development.required_length, 'right': hook.transverse_projection}
    )
    required_by_end = {item.end: development.required_length for item in preliminary_geometry.values()}

```

```

type_by_end = {item.end: 'straight' for item in preliminary_geometry}
for item in preliminary_geometry:
    if item.status == 'NG' and detailing['allow_standard_hooks'] and hook_f
        required_by_end[item.end] = hook.required_length
        type_by_end[item.end] = 'standard 90-degree hook'
tie_anchorage_geometries.extend(calculate_tie_end_anchorage_geometry(
    layer, tie_layer_positions[layer], 0.0, cap['length'], layout.bar_size,
    enclosure_bar_size=preliminary_design['crack_control_bar_size'],
    clear_cover=detailing['clear_cover'],
    required_development_lengths=required_by_end, anchorage_type=type_by_end
))
tie_development_table = records_to_dataframes({
    'tie_development': tension_development_records(tie_development_calculations
)})['tie_development']
tie_hook_table = records_to_dataframes({
    'tie_hooks': standard_hook_records([item[0] for item in tie_hook_alternativ
)})['tie_hooks']
tie_anchorage_table = records_to_dataframes({
    'tie_anchorage': tie_anchorage_records(tie_anchorage_geometries),
})['tie_anchorage']
print('\nAREMA 2.14 straight-bar development:')
for layer, item in tie_development_calculations.items():
    print(f' {layer}: basic={item.basic_length:.2f} in, confinement={item.conf
print('\nAREMA 2.17 standard 90-degree-hook alternatives:')
for layer, (item, fits) in tie_hook_alternatives.items():
    print(f' {layer}: required={item.required_length:.2f} in, transverse proje
print('\nAvailable end anchorage geometry:')
for item in tie_anchorage_geometries:
    print(f' {item.layer}, {item.end}: {item.available_length:.2f} in availabl
print(f"Layouts use {detailing['clear_cover']} in cover, a #{preliminary_design
print(f"Straight-bar requirements use AREMA 2.14; lightweight={detailing['light
print('STM longitudinal ties are scheduled as continuous bars without lap splic
print('End anchorage shown here is preliminary; the external-node cell replaces
else:
    print('Frame-boundary mode: bearing, anchorage, and member-capacity detailing c

```

Frame-boundary mode: bearing, anchorage, and member-capacity detailing checks are outside this validation scope.

```

In [12]: if run_design_checks:
    print('Strut capacity checks are deferred until both nodal-interface widths are
    print('No assumed default strut width is used in the production workflow.')
else:
    print('Frame-boundary mode: bearing, anchorage, and member-capacity detailing c

```

Frame-boundary mode: bearing, anchorage, and member-capacity detailing checks are outside this validation scope.

```

In [13]: if run_design_checks:
    nodal_group_geometries = {}
    nodal_face_checks = {}
    for combination, result in results.items():
        combination_geometries = []
        combination_checks = []
        for face in result.truss_model.external_faces:
            if face.kind == 'bearing_plate':
                source_id = combination_loads[combination][face.source_index]['id']

```

```

        location = f'load {source_id}'
        back_depth = preliminary_design['load_nodal_back_depth']
    else:
        source_id = piles[face.source_index]['id']
        location = f'{face.kind.replace("_head", "")} {source_id}'
        back_depth = preliminary_design['support_nodal_back_depth']
    geometry = construct_single_strut_external_nodal_zone(
        face, result.truss_model, result.member_forces, back_depth,
        label=location,
    )
    checks = check_subdivided_nodal_zone_faces(
        geometry, cap['width'], materials['fc_ksi'],
        phi=preliminary_design['phi_concrete'],
        stress_factors=preliminary_design['node_stress_factors'],
        confinement_factor=preliminary_design['nodal_confinement_factor'],
    )
    combination_geometries.append(geometry)
    combination_checks.extend(checks)
    nodal_group_geometries[combination] = tuple(combination_geometries)
    nodal_face_checks[combination] = tuple(combination_checks)

nodal_geometry_tables = {
    combination: records_to_dataframes(
        nodal_geometry_records(group_geometries=geometries)
    )['nodal_zone_groups']
    for combination, geometries in nodal_group_geometries.items()
}

nodal_check_tables = {
    combination: records_to_dataframes(
        design_check_records(nodal_face_checks=checks)
    )['nodal_face_checks']
    for combination, checks in nodal_face_checks.items()
}

print(f"{'combination':<24} {'location':<12} {'face':<17} {'demand':>9} {'width'
print('-' * 104)
for combination, checks in nodal_face_checks.items():
    for check in checks:
        print(f"{'combination':<24} {'check.group_label':<12} {'check.face_type':<17}
print('External nodal envelopes and bearing, back, and strut-interface faces ar
print('Any external node with multiple compression struts stops and requires an

# Replace midpoint anchorage proxies with the governing extended-strut/tie inte
extended_critical_sections = {}
extended_critical_sources = {}
for layer in common_tie_layouts:
    tie_y = cap['top_tie_y'] if layer.startswith('top') else cap['bottom_tie_y']
    intersections = {'left': [], 'right': []}
    for combination, geometries in nodal_group_geometries.items():
        face_kind = 'bearing_plate' if layer.startswith('top') else 'pile_head'
        candidates = [item for item in geometries if item.tributary.parent_kind
        if not candidates:
            continue
        for end, item in (('left', min(candidates, key=lambda value: value.trib
            coordinate = extended_strut_tie_critical_section(item, tie_y, end)
            intersections[end].append((coordinate, combination, item.label))
    if not all(intersections.values()):

```

```

        raise RuntimeError(f'No external extended-strut boundary found for {layer}')
    left = min(intersections['left'], key=lambda value: value[0])
    right = max(intersections['right'], key=lambda value: value[0])
    extended_critical_sections[layer] = {'left': left[0], 'right': right[0]}
    extended_critical_sources[layer] = {
        'left': f'extended strut boundary: {left[1]} / {left[2]}',
        'right': f'extended strut boundary: {right[1]} / {right[2]}',
    }
}
tie_anchorage_geometries = []
for layer, layout in common_tie_layouts.items():
    development = tie_development_calculations[layer]
    hook, hook_fits = tie_hook_alternatives[layer]
    straight = calculate_tie_end_anchorage_geometry(
        layer, tie_layer_positions[layer], 0.0, cap['length'], layout.bar_size,
        enclosure_bar_size=preliminary_design['crack_control_bar_size'],
        clear_cover=detailing['clear_cover'],
        required_development_lengths={'left': development.required_length, 'right': development.required_length},
        critical_section_coordinates=extended_critical_sections[layer],
        critical_section_basis=extended_critical_sources[layer],
    )
    required_by_end = {item.end: development.required_length for item in straight}
    type_by_end = {item.end: 'straight' for item in straight}
    for item in straight:
        if item.status == 'NG' and detailing['allow_standard_hooks'] and hook_fits:
            required_by_end[item.end] = hook.required_length
            type_by_end[item.end] = 'standard 90-degree hook'
    tie_anchorage_geometries.extend(calculate_tie_end_anchorage_geometry(
        layer, tie_layer_positions[layer], 0.0, cap['length'], layout.bar_size,
        enclosure_bar_size=preliminary_design['crack_control_bar_size'],
        clear_cover=detailing['clear_cover'],
        required_development_lengths=required_by_end, anchorage_type=type_by_end,
        critical_section_coordinates=extended_critical_sections[layer],
        critical_section_basis=extended_critical_sources[layer],
    ))
tie_anchorage_table = records_to_dataframes({'tie_anchorage': tie_anchorage_records})
cage_reviews = []
for layer, layout in common_tie_layouts.items():
    layer_anchorage = [item for item in tie_anchorage_geometries if item.layer == layer]
    selected_hooks = [tie_hook_alternatives[layer][0].transverse_projection for item in layer_anchorage]
    cage_reviews.append(check_reinforcement_cage_congestion(
        layer, layout, member_depth=cap['depth'],
        crack_control_bar_size=development_grid.bar_size,
        crack_control_spacing=development_grid.selected_spacing,
        maximum_aggregate_size=preliminary_design['maximum_aggregate_size'],
        hook_projections=selected_hooks,
        anchorage_statuses=[item.status for item in layer_anchorage],
    ))
cage_review_table = records_to_dataframes({'cage_reviews': reinforcement_cage_reviews})
print('\nFinal extended-strut anchorage and cage review:')
for item in tie_anchorage_geometries:
    print(f' {item.layer}, {item.end}: critical x={item.critical_section_coordinates}')
for item in cage_reviews:
    print(f' {item.layer}: longitudinal clear {item.provided_transverse_clear_cover}')
    print('Maximum aggregate size and cover are visible inputs. Section-level spacing')
else:
    print('Frame-boundary mode: bearing, anchorage, and member-capacity detailing c
```

Frame-boundary mode: bearing, anchorage, and member-capacity detailing checks are outside this validation scope.

```
In [14]: if run_design_checks:
    interior_nodal_group_geometries = {}
    interior_nodal_geometry_failures = {}
    for combination, result in results.items():
        geometries = []
        failures = []
        external_node_ids = {face.node_id for face in result.truss_model.external_faces}
        for node_id, point in enumerate(result.truss_model.nodes):
            if node_id in external_node_ids:
                continue
            connected = [
                result.member_forces[member_id]
                for member_id, member in enumerate(result.truss_model.members)
                if node_id in member
            ]
            if not any(force < -1e-9 for force in connected):
                continue
            if np.isclose(point[1], cap['top_tie_y']):
                back_depth = preliminary_design['load_nodal_back_depth']
            elif np.isclose(point[1], cap['bottom_tie_y']):
                back_depth = preliminary_design['support_nodal_back_depth']
            else:
                failures.append((node_id, 'node is outside a defined tie layer'))
                continue
            try:
                geometries.extend(construct_continuous_tie_nodal_zone(
                    result.truss_model, result.member_forces, node_id, back_depth,
                    assume_continuous_reinforcement=True,
                    split_opposite_sides=True,
                    tie_axis=(1.0, 0.0),
                    tie_boundary_points=((0.0, point[1]), (cap['length'], point[1]))
                ))
            except ValueError as exc:
                failures.append((node_id, str(exc)))
        interior_nodal_group_geometries[combination] = tuple(geometries)
        interior_nodal_geometry_failures[combination] = tuple(failures)

    interior_nodal_geometry_tables = {
        combination: records_to_dataframes(
            nodal_geometry_records(group_geometries=geometries)
        )['nodal_zone_groups']
        for combination, geometries in interior_nodal_group_geometries.items()
    }

    interior_nodal_face_checks = {}
    for combination, geometries in interior_nodal_group_geometries.items():
        interior_nodal_face_checks[combination] = tuple(
            check for geometry in geometries
            for check in check_interior_nodal_zone_faces(
                geometry, cap['width'], materials['fc_ksi'],
                phi=preliminary_design['phi_concrete'],
                stress_factors=preliminary_design['node_stress_factors'],
                confinement_factor=1.0,
            )
        )
```

```

    )
    for combination in results:
        print(f'{combination}: constructed {len(interior_nodal_group_geometries[combination])}
              checks = interior_nodal_face_checks[combination]
              print(f' capacity faces: {len(checks)} checked; maximum D/C={max((item.dc_
              for node_id, reason in interior_nodal_geometry_failures[combination]:
                  print(f' unresolved node {node_id}: {reason}')
        print('Interior policy: continuous horizontal ties, midpoint tributaries, separate
    else:
        print('Frame-boundary mode: bearing, anchorage, and member-capacity detailing c

```

Frame-boundary mode: bearing, anchorage, and member-capacity detailing checks are outside this validation scope.

```

In [15]: if run_design_checks:
    strut_width_profiles = {}
    strut_width_tables = {}
    geometry_strut_checks = {}
    for combination, result in results.items():
        profiles = calculate_strut_width_profiles(
            result.truss_model, result.member_forces,
            nodal_group_geometries[combination]
            + interior_nodal_group_geometries[combination],
        )
        strut_width_profiles[combination] = profiles
        strut_width_tables[combination] = records_to_dataframes({
            'strut_widths': strut_width_records(profiles),
        })['strut_widths']
        geometry_strut_checks[combination] = tuple(
            check_strut(
                profile.member_id, result.member_forces[profile.member_id],
                profile.controlling_width, cap['width'], materials['fc_ksi'],
                eps_1=preliminary_design['strut_eps_1'],
                phi=preliminary_design['phi_concrete'],
                fcu_cap_factor=preliminary_design['strut_fcu_cap_factor'],
            )
            for profile in profiles if profile.status == 'complete'
        )

    print(f"{'combination':<24} {'member':>7} {'end i':>9} {'end j':>9} {'control':>9}")
    print('-' * 86)
    for combination, profiles in strut_width_profiles.items():
        for profile in profiles:
            wi = '-' if profile.width_i is None else f'{profile.width_i:.2f}'
            wj = '-' if profile.width_j is None else f'{profile.width_j:.2f}'
            wc = '-' if profile.controlling_width is None else f'{profile.controlling_width:.2f}'
            print(f"{'combination':<24} {'profile.member_id':>7d} {'wi':>9} {'wj':>9} {'wc':>9}")
    print('Only complete two-ended profiles are used for geometry-derived strut capacity')
    print('Partial or missing profiles require construction of the unresolved interior')
    for combination, checks in geometry_strut_checks.items():
        max_dc = max((check.dc_ratio for check in checks), default=float('nan'))
        print(f'{combination}: {len(checks)} geometry-width strut checks; maximum D/C={max_dc}')
    else:
        print('Frame-boundary mode: bearing, anchorage, and member-capacity detailing c

```

Frame-boundary mode: bearing, anchorage, and member-capacity detailing checks are outside this validation scope.

```
In [16]: if run_design_checks:
    # Optional reviewed field evidence. Do not label bottle/fan behavior from unequ
    strut_shape_evidence = {
        combination: {'midspan_widths': {}, 'fan_member_ids': ()}
        for combination in results
    }
    strut_shape_classifications = {}
    strut_shape_tables = {}
    conservative_strut_designs = {}
    conservative_strut_design_tables = {}
    for combination, profiles in strut_width_profiles.items():
        evidence = strut_shape_evidence[combination]
        classifications = classify_strut_shapes(
            profiles, midspan_widths=evidence['midspan_widths'],
            fan_member_ids=evidence['fan_member_ids'],
        )
        strut_shape_classifications[combination] = classifications
        designs = apply_conservative_strut_policy(profiles, classifications)
        conservative_strut_designs[combination] = designs
        strut_shape_tables[combination] = records_to_dataframes({
            'strut_shapes': strut_shape_records(classifications),
        })['strut_shapes']
        conservative_strut_design_tables[combination] = records_to_dataframes({
            'conservative_strut_designs': conservative_strut_design_records(designs)
        })['conservative_strut_designs']
        counts = {}
        for item in classifications:
            counts[item.shape] = counts.get(item.shape, 0) + 1
        print(f'{combination}: {counts}')
    print('Simplified policy: minimum end width controls; every non-prismatic strut
    crack_control_table = records_to_dataframes({
        'crack_control': crack_control_records(crack_control_grid),
    })['crack_control']
    for grid in crack_control_grid:
        description = (f'{grid.legs}-leg #{grid.bar_size}' if grid.bar_size is not
        print(f'{grid.direction}: {description} at {grid.selected_spacing:.1f} in;

    calculation_records = pier_cap_analysis_records(
        common_analysis,
        external_nodal_checks=nodal_face_checks,
        interior_nodal_checks=interior_nodal_face_checks,
        strut_checks=geometry_strut_checks,
        tie_checks=[check for _, _ in tie_checks],
        tie_layouts=common_tie_layouts.values(),
        anchorage=tie_anchorage_geometries, cage_reviews=cage_reviews,
        crack_control=crack_control_grid,
    )
    calculation_metadata = build_calculation_metadata(
        inputs={
            'project': project, 'cap': cap, 'materials': materials,
            'detailing': detailing, 'code_profile': code_profile,
            'piles': piles, 'load_components': load_components,
            'load_combinations': load_combinations,
```

```

        'provided_reinforcement': provided_reinforcement,
    },
    results=calculation_records, project_revision=project['revision'],
    repo_root=repo_root,
)
calculation_records['report_metadata'] = [calculation_metadata]
calculation_tables = records_to_dataframes(calculation_records)
calculation_report = build_pier_cap_calculation_report(
    project=project, cap=cap, materials=materials, detailing=detailing,
    code_profile=code_profile, common_analysis=common_analysis,
    tie_layouts=common_tie_layouts, anchorage=tie_anchorage_geometries,
    cage_reviews=cage_reviews, strut_checks=geometry_strut_checks,
    external_nodal_checks=nodal_face_checks,
    interior_nodal_checks=interior_nodal_face_checks,
    metadata=calculation_metadata,
)
html_report = build_pier_cap_html_report(
    project=project, cap=cap, materials=materials, detailing=detailing,
    code_profile=code_profile, common_analysis=common_analysis,
    tie_layouts=common_tie_layouts, anchorage=tie_anchorage_geometries,
    cage_reviews=cage_reviews, strut_checks=geometry_strut_checks,
    external_nodal_checks=nodal_face_checks,
    interior_nodal_checks=interior_nodal_face_checks,
    calculation_records=calculation_records, crack_control=crack_control_grid,
    reinforcement_source=('provided' if any(key in (provided_reinforcement or {
    metadata=calculation_metadata,
)
display(calculation_tables['model_quality'])
print(calculation_report)
if output_options['write_files']:
    output_directory = Path(output_options['directory'])
    export_csv_tables(calculation_records, output_directory / 'csv')
    export_json_tables(calculation_records, output_directory / 'pier_cap_result')
    write_calculation_report(calculation_report, output_directory / 'calculation_report.json')
    write_html_report(html_report, output_directory / 'pier_cap_report.html')
    if output_options['write_excel']:
        export_excel_tables(calculation_records, output_directory / 'pier_cap_result.xlsx')
    print(f'Calculation files written to {output_directory.resolve()}')
else:
    print('Frame-boundary mode: bearing, anchorage, and member-capacity detailing c

```

Frame-boundary mode: bearing, anchorage, and member-capacity detailing checks are outside this validation scope.

6. Analysis status and scope

The analysis checks below are automatic. Project activities outside this two-dimensional STM are identified as scope boundaries, not unresolved analysis failures.

Gate	Current status	Required development
Common topology across combinations	Implemented	One pruned candidate structure; all cases resolved; reversals reported and designed for both signs

Gate	Current status	Required development
Non-nodal member crossings	Implemented	Diagonal webs default to compression-only; remaining intersections stop the workflow
Applied loads	User supplied	Railroad load generation and selection of applicable combinations are outside scope
Tie reinforcement	Implemented	Automatic common schedule, spacing, development, hooks, and section-level compatibility
Strut shape/capacity	Implemented conservatively	Minimum end width; bottle detailing for non-prismatic struts; no fan credit
Crack-control grid	Implemented	AREMA 0.003 ratios, spacing ceiling, and aggregate spacing
External nodal zones	Implemented	Face-specific AREMA 2025 factors; explicit plan required only for a true multi-strut external node
Interior nodal zones	Implemented conservatively	Continuous ties, midpoint tributaries, separated sides, no confinement enhancement, all faces checked
Foundation response	Outside scope	Reactions may be imported from a compatible foundation/global model
Final bar positioning and drawings	Outside scope	The notebook completes section-level checks only
AREMA resistance factors	Implemented	Article 2.30.2b: 0.70 compression and 0.90 reinforced-concrete STM tension

The notebook reports all applied code factors, visible project defaults, governing checks, and scope boundaries in the calculation summary.

7. Analysis record

For a project calculation package, retain the completed input cell, governing AREMA references, validation output, topology plots, force envelopes, manual topology decisions, and all future design-check results.